

Diseño y Análisis de Algoritmos

Prueba Sumativa 1

Profesor: Felipe Ruiz

2 de Octubre, 2025

Nombre: _____ Rut: _____

Indicaciones

- Puntaje total: **90 puntos**. Duración: **90 min**. Entrega de resultados: **23 de Oct, 2025**.
- Esta evaluación tiene 4 páginas (incluyendo la portada) y 3 preguntas. Compruebe que dispone de todas las páginas. Responda a las preguntas en los espacios previstos para ello en las hojas de preguntas. Asegúrese de marcar apropiadamente sus respuestas.
- Durante la prueba no se puede utilizar: teléfono móvil, apuntes y/o computador. Está prohibido intentar conectarse a internet de cualquier manera. Si es sorprendido obtendrá la calificación mínima. Tampoco puede utilizar dispositivos de almacenamiento externos o cualquier otro dispositivo, relojes inteligentes, etc.
- Lea la prueba completamente **DOS** veces antes de hacer cualquier pregunta.
- Una prueba respondida correctamente en un 60 %, de acuerdo con las ponderaciones asignadas, corresponde a una nota 4,0.
- Solamente se pueden realizar preguntas durante los primeros 10 minutos de la prueba. Solo se responderán preguntas respecto a los enunciados a viva voz.
- La prueba es individual; cualquier sospecha de copia será calificada con la nota mínima y el caso será remitido al comité de ética.
- En su espacio personal no debe haber nada más que hojas de papel en blanco, lápiz, goma y/o calculadora.
- El resto de sus implementos debe guardarlos dentro de su mochila/bolso y ésta debe posicionarse al frente debajo de la pizarra. *Si leyó hasta este punto, felicidades: para saber que lo hizo, dibuje una estrella al final de esta página.*

Acepto las condiciones firmando: _____

TriMergeSort**(30 puntos)**

Considere el siguiente esquema de algoritmo *TriMergeSort*: dados n elementos, *divide* el arreglo en 3 partes de igual tamaño, *ordena* recursivamente cada parte y luego *combina* las tres listas ordenadas en tiempo $O(n)$ (puede suponer que la combinación triple se hace de manera lineal). Suponga que n es potencia de 3 y que $T(n) = \Theta(1)$ para $n \leq n_0$, donde n_0 es una constante.

- (a) Modele la ecuación de recurrencia $T(n)$ para el tiempo de ejecución total. **(10 puntos)**
- (b) Resuelva por expansión la recurrencia del apartado (a). Muestre expansiones, identifique el patrón, elija t adecuado y concluya la complejidad temporal en notación $\Theta(\cdot)$. **(20 puntos)**

Ballenas en el Archipiélago de Humboldt**(30 puntos)**

En el **Archipiélago de Humboldt** se registró durante n días consecutivos la **cantidad de ballenas** observadas en un censo diario, almacenando los valores en un arreglo $A[1..n]$. Al comienzo, el conteo **crecía cada día**; posteriormente, coincidente con el inicio de un proyecto minero en la zona, el conteo **comenzó a disminuir** cada día. En otras palabras, existe un día p tal que para todo $i \in \{1, \dots, p-1\}$ se cumple $A[i+1] > A[i]$ y para todo $i \in \{p, \dots, n-1\}$ se cumple $A[i+1] < A[i]$:

$$A[1] < A[2] < \dots < A[p] \quad \text{y} \quad A[p] > A[p+1] > \dots > A[n]$$

- (a) Diseñar un algoritmo que identifique el índice p (día en que el conteo alcanza su máximo) y que se ejecute en $O(\log n)$ de tiempo, *cuyo criterio de avance se base en comparaciones entre elementos de A* , evitando accesos fuera de rango. **(25 puntos)**
- Describir la idea del algoritmo en palabras (pseudocódigo opcional).
 - Ser explícito sobre las comparaciones que guían el avance (qué elementos se comparan y cómo decide a qué subintervalo continuar).
 - Justificar brevemente la complejidad $O(\log n)$.
- (b) Describa las comparaciones exactas y el comportamiento cuando el máximo está en la *primera* ($p = 1$) o *última* ($p = n$) posición; indique qué valor retorna su algoritmo en cada caso y explique por qué agregar estas verificaciones preservan la complejidad asintótica $O(\log n)$. **(5 puntos)**

Línea 5 — Operación *express* del Metro de Santiago**(30 puntos)**

En la **Línea 5** del Metro de Santiago, durante horas punta se puede activar un patrón *express* para mejorar la fluidez. Para evitar **cuellos de botella** y tiempos de detención elevados, el tren **no puede detenerse en estaciones adyacentes** cuando opera en modo *express*. Para cada estación i se estima una demanda atendida $P[i] \geq 0$ si el tren *sí* se detiene allí. Dado $P[1..n]$ (estaciones en orden lineal), determine la **cantidad máxima de pasajeros** que puede atender el tren en un recorrido bajo esta restricción.

- (a) Para el ejemplo

$$P[1..6] = \langle 15, 8, 12, 20, 4, 10 \rangle,$$

calcule **a mano** el valor óptimo para cada prefijo $F(i)$ (máximo atendido usando solo las primeras i estaciones) para $i = 1, 2, \dots, 6$, e indique el valor final $F(6)$.

Complete además los **casos base**: $F(0) = \underline{\hspace{1cm}}$, $F(1) = \underline{\hspace{1cm}}$. **(5 puntos)**

i	1	2	3	4	5	6
$F(i)$						

- (b) Complete los espacios en blanco de la siguiente definición de $F(i)$, donde $F(i)$ representa el máximo de pasajeros atendidos usando solo las primeras i estaciones y sin detenerse en estaciones adyacentes. **(10 puntos)**

$$F(i) = \begin{cases} \underline{\hspace{1cm}}, & \text{si } i = 0, \\ \underline{\hspace{1cm}}, & \text{si } i = 1, \\ \text{máx}\{ \underline{\hspace{1cm}}, \underline{\hspace{1cm}} \}, & \text{si } 2 \leq i \leq n, \end{cases} \quad \text{con } i \in \{0, 1, \dots, n\}.$$

- (c) Usando pseudocódigo, implemente $\text{MAXPASSENGERS}(P[1..n], n)$ que retorne el máximo de pasajeros atendidos hasta la estación n bajo la restricción de no detenerse en adyacentes. Puede usar *Top-Down con memoización* o *Bottom-Up*, en $O(n)$ tiempo y $O(n)$ espacio adicional. Agregue una **justificación breve** de por qué su procedimiento cumple las cotas de complejidad solicitadas. **(15 puntos)**